OMNI_RAG

CS5004
FINAL PROJECT
Professor: Dr. Rached Zantout

Nguyen "John" Ha
Rui "Snow" Song
Siyuan "Ash" Liang

Northeastern University

https://github.com/johnha912/rag-core-engine

INTRODUCTION

The **Omni RAG Core Engine** is a Java 21 and Spring Boot-based system that transforms diverse documents into a searchable knowledge base, providing hallucination-free AI answers with precise citations.

OBJECTIVES

Build a robust RAG engine using modern Java 21 & Spring Boot. Enable seamless document uploads and natural language querying with verifiable source citations.

DESIGN GOAL

Deliver hallucination-free "open-book" answers. Utilize domain-aware chunking (scripts by scenes, legal by clauses) combined with semantic vector search.

APPLICATION

Functions as an accurate knowledge base for specialized fields: film production documents, legal texts, and structured technical documentation.

TECHNICAL STACK

CORE FRAMEWORK

Java 21 | Spring Boot 3.2.5 (Web + Actuator)
Maven 3.9+ (Build)
OkHttp 4.12.0 (30s/120s timeouts)
Gson 2.10.1 (JSON + SSE encoding)

STORAGE

SQLite JDBC 3.45.3.0 (Vector BLOB Storage)
CopyOnWriteArrayList (In-Memory Fallback)
ConcurrentHashMap (Embedding Cache)

AI MODELS

OpenAI GPT-4o-mini (Gen + Re-ranker)
OpenAI text-embedding-3-small (1536-dim)
Cosine Similarity (Hand-implemented)

PARSING & EXTRACTION

PDFBox (.pdf) | POI (.docx) | Jsoup (.html)
Java Adapters: .txt, .md, .csv, .json, .vtt, .srt, .fdx

TESTING

JUnit 5.10.0 + Spring Boot Test

LIVE OPERATION

Upload, Index, and Query

Upload
.pdf/.md/.srt/e.g.

Real-time vector
indexing

Ask question within
server-sent events
streaming answers

LIVE OPERATION

01
User Input
Upload & Query: `RagController` REST API

02
Orchestration
Pipeline Coordinator: `RagOrchestrator`

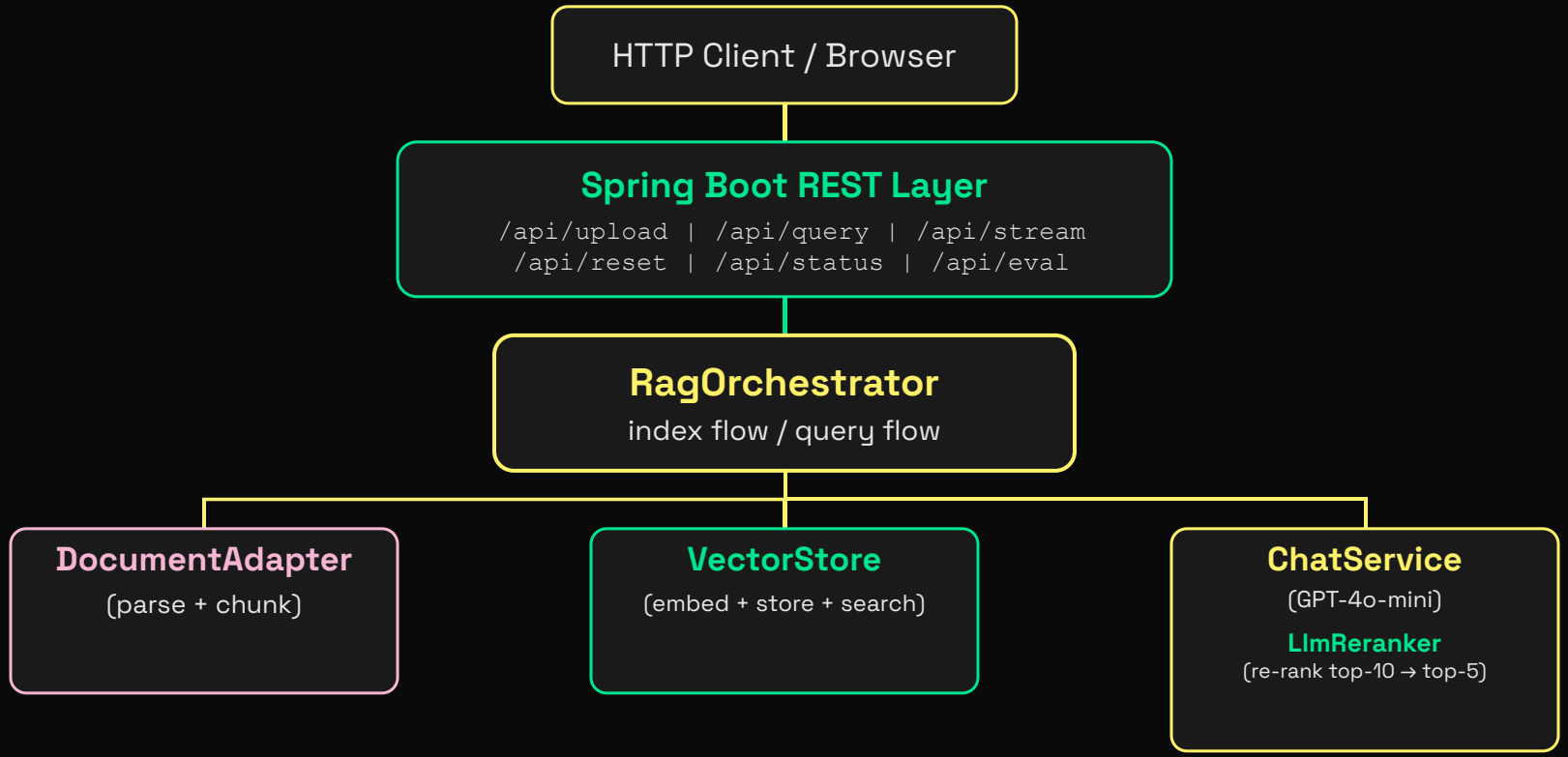
03
Extraction
Document Parsing: `DocumentAdapter`

04
Vectorization
`EmbeddingServiceImpl` - Text to float arrays

05
Storage
`SQLiteVectorStore`: Cosine similarity search

06
Output
`LlmReranker` & `ChatService`: Refines Top-10 to Top-5, then Answers with citations

ARCHITECTURE



Index flow: upload → DocumentAdapter (parse + chunk) → VectorStore (embed + store)

Query flow: question → VectorStore (search top-10) → LlmReranker (re-rank → top-5) → ChatService (generate) → answer + sources

THE ABSTRACTION

Orchestrator's Reality

The coordinator operates in a "pure" domain. It has no idea OpenAI or SQLite exist.

`embeddingService.embed(text)`

Input: Raw String → Output: Vector

`vectorStore.search(query, topK)`

Input: Vector → Output: List<Document>

Implementation "Noise"

Hidden beneath the interfaces is the messy infrastructure:

- OkHttp 4.12.0: Network retries & TLS
- Gson: parsing `choices[0].delta`
- SQLite JDBC: Batch inserts & WAL
- Math: `cos() = (A B) / (|A||B|)`

The orchestrator only knows about text and vectors.

API WORKFLOW & CAPABILITIES

01. INGESTION & DOMAIN BINDING

`POST /api/upload [file: screenplay.pdf, domain: film script]`

`Response: { name: "screenplay.pdf", chunks: 142 }`

Batch-embedded: 142 chunks in 2 OpenAI calls (sub-batched @100, cache-aware)

02. SEMANTIC QUERY

`POST /api/query ["What happens in the second act?"]`

`Answer: "The protagonist encounters..."`

03. MULTI-TURN RESOLUTION

`Follow-up: "Tell me more about that scene."`

System resolves "that scene" to Scene 12 using memory history.

04. DOMAIN HOT-SWAP

`DELETE /api/reset POST /api/upload [lease.pdf]`

`Query: "Heater broken?" Persona: Legal Assistant`

05. EVALUATION ENDPOINT

`POST /api/eval { retrieval_hit_rate: 0.94, keyword_accuracy: 0.88 }`

06. STREAMING ANSWERS

`GET /api/stream?question=... event: "The" ... event: [DONE]`

Tokens encoded with Gson safe for every control character.

ENGINEERING STACK

- **Framework:** Spring Boot 3.2, Java 21
- **Intelligence:** GPT-4o-mini, text-embedding-3 (batch API)
- **Storage:** SQLite via JDBC
- **Parsing:** PDFBox, POI, Jsoup, Gson
- **Client:** OkHttp 4.12 (30s/120s/30s timeouts)
- **Testing:** JUnit 5, Spring Test

Project Core
Code Walkthrough

PATTERNS

Inheritance & Template Method

`BaseDocumentAdapter` handles parsing logic for PDF, DOCX, and MD once.

Subclasses focus only on `buildChunks()`:

- **ScriptAdapter:** Scene-boundary splitting
- **RentalLawAdapter:** Section-number splitting
- **GeneralAdapter:** Fixed-size w/ overlap

Interface & Polymorphism

`VectorStore` interface decouples the core logic from storage details.

`@Primary SQLiteVectorStore`

Persistent storage is swapped in for `InMemoryVectorStore` with zero code changes in the Orchestrator.

OmniRAG LIMITATIONS

01. FILE SIZE CAPACITY

`max-file-size: 50MB`

Hard cap configured in application properties. Requests exceeding 50MB are rejected at network layer.

02. ACCEPTED FORMATS

PDF, DOCX, MD, SRT, VTT, TXT, HTML, CSV, JSON, FDX

Strict whitelist; unsupported types trigger 400 IllegalArgumentException.

03. EMBEDDING LIMITS

`Batch: 300,000 Tokens`

Groups of 100 chunks must stay under OpenAI's 300k limit. Minimal risk for standard chunking.

04. AUTH & RATE LIMITING

No login, no rate limits, no per-user quotas. Accessible for anonymous query/upload.

Design Priority: Open Access > Security

05. SERVER ENVIRONMENT

JDK 21 permanently added to replit.nix to mitigate Maven reset issues.

`replit.nix: pkgs.jdk21, pkgs.maven`

REFLECTIONS & CONCLUSIONS

The "Invisible" Mutation Bug

Mutating defensive copies of metadata.
`chunk.getMetadata().put("domain", val)`

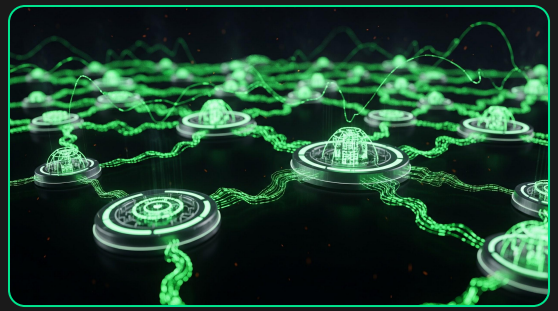
Result: Domain was never saved.

The Fix: Dedicated write path. Reinforced encapsulation.

The Reinvented-Wheel Bug

Hand-rolled JSON escaping for control chars `\u0000-\u001F` crashed parser.

The Fix: `gson.toJson(token)`. Don't reimplement a library's spec.



The Interface Payoff

Freezing `VectorStore` enabled parallel dev. SQLite swap in Wk 3 required **zero changes**. `EmbeddingService` evolved via default methods without breaking callers.

Full-Spectrum Application

- Interfaces & Polymorphism
- Default Methods & Evolution
- Inheritance & Template Method
- Encapsulation & DI

LESSON: Controllers handle transport, not serialization. OOD decoupling pays off.

THANK YOU!

Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. https://arxiv.org/abs/2005.11401
OpenAI. (2024). Embeddings API — text-embedding-3-small. https://platform.openai.com/docs/guides/embeddings
Spring Framework. (2024). Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/docs/current/reference/html/